PrimePass: Event Ticketing System

Serden-Yilmaz Kose, Niklas Raesalmi, MD.Nur-E Ferdaus, Huan Vo, Niranjan Sreegith

***Abstract*—PrimePass is a distributed ticket selling system designed to support real-time event browsing and ticket purchasing under concurrent user demand. The system is deployed on Kubernetes and consists of a web frontend, a Flask-based backend API, and a PostgreSQL database configured with a primary and a replica. The main idea is preventing overselling when multiple users purchase the same ticket type simultaneously. PrimePass addresses this by using a database update that only confirms a purchase when sufficient tickets remain. This makes sure that the inventory never becomes negative. To evaluate the implementation, we run functional tests and workload-based experiments that measure response latency and payload effects under different request volumes and response sizes. The results demonstrate that the system maintains correctness under concurrent purchase attempts and remains responsive for browsing and purchasing.**

Link to the Final Demo Video: https://youtu.be/H5lC2QrodfQLink

https://github.com/Serden-YilmazKose/PrimePass/tree/main

***Index Terms*—Distributed, REST API, Flask, MariaDB**

I. Problem Statement

Online ticket sales are a classic activity where many users may attempt to purchase a limited number of tickets concurrently, and the system must remain correct (no overselling) while still providing acceptable performance. A naive implementation can confirm too many purchases when requests race, leading to inconsistent inventory state and failed customer experience. In addition, the system must be deployable in a distributed environment where components run on separate nodes and communicate over the network, and it should tolerate common operational events such as pod restarts or scaling. The goal of this project is to design and implement a distributed ticketing application that provides event browsing, user login/registration, ticket purchasing, and order viewing through an API and web UI, guarantees correctness under concurrent purchase requests, and can be deployed and reproduced on Kubernetes with a multi-node architecture. As required, the project must also include a measurable evaluation with at least two scenarios to measure performance and demonstrate the system's behavior under different loads or payload sizes.

II. System Design

A. RESTful Application Programming Interface & PostgreSQL Server

1) Architectural Overview: Whenever a user performs an action, the system must handle the HTTP request. There are several different types of actions a user can take: registering an account, logging into an existing account, view events, purchase tickets, and view purchased tickets. The following will show the most crucial tools used to achieve this, and a brief explanation to their relevance in the project.

- Flask is a web framework designed for Python that is designed to allow for simple web applications, including RESTful Application Programming Interface (API). The application we designed handles the HTTP GET/POST requests made by the user, to handle the aforementioned specified cases.
- PostgreSQL is an object-relational database management system, which is designed for reliability, feature robustness, and performance. In our project, it is used to store user data, including name, email, and a password, along with details regarding events and ticketing.

2) Database Schema Design: The structure of the database, otherwise known as the database schema design, is composed of the following tables:

- USER: Unique users, identified by name, email, and hashed password
- EVENT: Events and their data, such as time, venue, etc...
- TICKET: Tracks the different kinds of tickets, along with capacity of ticket sales and remaining tickets left.
- ORDER: A table to show all ticket sales, which are linked to the users and event.

III. Implementation

A. RESTful API

The design of the RESTFul API and PostgreSQL relational database management servers was briefly explained in a previous section of this paper. The following sections will elaborate and add detail to the logic behind the different frameworks and tools used for this section of the project.

1) ***User Registration and Login****:* Upon a POST request to `/api/login`, the Flask application will check if the request made was provided in a JSON format. If it isn't, an error code is returned informing the user that a JSON format is required. Otherwise, the the name, email, and password will be extracted. An error code will be return early if no email nor password was provided. Then, the application will make a query to the PostgreSQL server to fetch the ID and stored password hash from the `USER`. If the user already exists in the database, the following will happen: the provided password will be hashed, and if the stored password hash matches with the one provided, a successful 200 status code will be provided, and the user will be allowed to enter. Otherwise, an error is returned, informing the user about wrong credentials.

New users will be able to register an account using registration endpoint provided by the RESTful API. Users provide their name, email and password using the frontend interface. The backend then authenticates the input information and verifies the existence of the email address. All users are given a unique identifier (UUID) which is employed in the system to identify the user in a future interactions.

The log in feature enables registered users to identify themselves. In case of a successful authentication, the backend issues a unique user identifier. This identifier is then stored in the frontend and utilized in the case of placing ordering requests like the purchase of tickets or recording activities.

*2) **Viewing Events**:* When a GET request is made to `/api/events`, the application will make a `SELECT` statement from the `EVENT` table to fetch the following data from the database server: event ID, title, venue, city, starting time, ending time, status, and remaining available tickets. Then, it formats the fetched data into a JSON array, and will return it. It is safe to assume that nothing can go wrong here, since all the endpoint is responsible for is fetching existing events. If none exists, it will simply return nothing.

*3) **Purchasing Tickets**:* When a POST request is made to `/api/purchase`, the application will first verify if the request was made in a JSON format. If it wasn't, an error will be returned informing the user that a JSON format is reuqired. Otherwise, it extracts the following data from the request: user ID, ticket ID, and quantity. If any one of those were not provided, an error is returned to inform the user that all fields are required to purchase a ticket. Moreover, if the quantity is fewer than or equal to zero, another error is returned to inform the user of the invalid quantity. If there are enough available tickets left, an `UPDATE` query is made. However, if no tickets are available, an error is returned to inform the user that all tickets have been sold out. Then, a loop is performed to add the new purchased tickets into the `ORDER` table using a `INSERT` statement. A successful status will be returned to the user. If anything fails during either the `UPDATE` or `INSERT` statements, an error will be returned to inform the user that the transaction has failed.

*4) **View Purchased Tickets**:* When a GET request is made to `/api/orders`, the application will verify if the user ID is given as a parameter. If it wasn't, the application will return an error to inform the user that the user ID was missing from the parameters. Otherwise, the applicaton will make a `SELECT` query to the database to retrieve the following data: order ID, status, created at time, name, price, title, city, and starting time. Then, similar to the viewing events endpoint, it will format the data into a JSON array, and return it to the user. If the user has no purchased orders, it will return nothing.

*5) **Activity Tracking**:* User interactions with the platform are logged in the system to monitor system usage and analyze user behavior. The viewing is an activity that is submitted to the backend via the endpoint `/api/ activity`

Each activity record consists of user identifier, event identifier, the type of the action performed and the optional metadata. Such records are saved in the database and later may be utilized in analytics or recommendation system.

*6) **Distributed Tracing**:* The system applies distributed tracing with the help of trace identifiers. In every received request, a trace ID can be attached via the HTTP header `X-Trace-ID`. In case the header is absent, the system creates a new distinct identifier.

Trace ID is added to the log messages so that one can track individual requests through multiple operations. This makes debugging easier and it allows one to see how requests are being handled in the backend.

*7) **Logging System**:* The backend uses a structured logging system to enhance debugging and monitoring of the system. The logging is configured with the help of the Python logging library and logs the key events of the system including API calls, user authentication and ticket purchasing actions.

Every log entry contains a timestamp, a log level and a message describing the event. This logging mechanism helps in identifying system issues and checking the application behavior when it is running.

Error Handling and Response Codes The API utilizes the standard HTTP status codes, as described with the following:

- 200-210: Success.
- 400: Error on the client.
- 401: Failure involving authentication.
- 500: Exception of transaction failure on the server.

B. PostgreSQL Server

The database is built on PostgreSQL, which was chosen for the reliability and performance benefits. Moreover, PostgreSQL has Python libraries to make it simpler to connect to the server from our Flask application and docker images such as the Bitnami PostgreSQL image that comes with built-in, pre-configured replication support that makes setup much simpler. The following subsections will describe the database schema and explain the structure of the four tables: `USER`, `EVENT`, `TICKET`, and `ORDER`.

1) USER Table: The `USER` table is responsible for storing the users' names, emails, and ID numbers. We decided to use universally unique identifiers (UUID), for their low collision occurrence. Thus, the ID is stored as a primary key, and is a `VARCHAR(36)`. Moreover, to avoid duplicated accounts from being registered, we set the `email` column as `UNIQUE`.

2) EVENT Table: The `EVENT` table is responsible for storing information regarding relevant events. The primary key would be the ID, which is stored as an `INT`. The rest of the table holds the following data: `title\, \verb`venue—, `city`, `description`, `starts_at`, `ends_at`, `status`, and `created_at`. To avoid issues with timezones, although this project was meant to be implemented in a rather small country, we opted to set the time-based columns as `DATETIME`.

3) TICKET Table: The `TICKET` table is responsible for storing the inventory and pricing for events. Again, the primary key is ID. The table then stores the following data: `name`, `price`, `capacity`, and `remaining`. Since both the capacity and remaining integers are stored in the same table, this makes it easier to evaluate if any available tickets remain, or if they have all been sold out. This table hold a foreign key, which is the event ID from the `EVENT` table.

4) ORDER Table: Finally, the `ORDER` table is responsible to act as a ledger to show all successful transactions of tickets. It stors the following data: `id`, `user_id`, `ticket_id`, `status`, and `created_at`. Naturally, the `user_id` and `ticket_id` columns are foreign keys to the `USER` and `TICKET` tables, respectively. Despite `ORDER` already being

a keyword in structured query language (SQL), we were able to use it as a table name by using backticks: `ORDER`.

IV. Results

Figure 1 shows the main user interface of the application. From this screen, users can view available ticket inventory and purchase tickets. The backend that recieves the commands handles ticket validation and transactions.

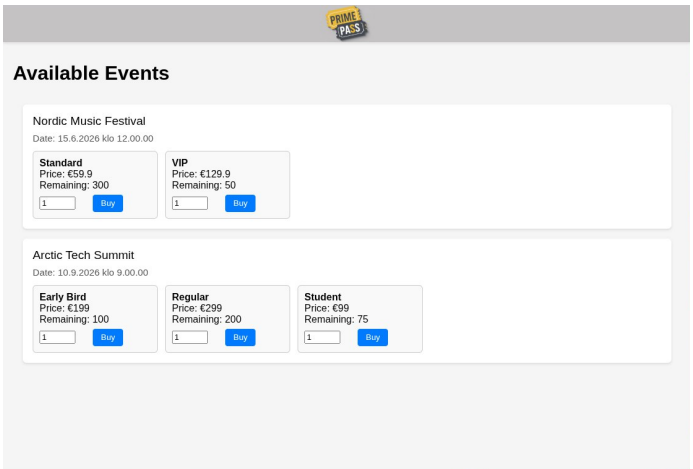

Fig. 1. Frontend home screen of the application.

When a purchase request is valid and tickets are available, the backend processes the transaction. The frontend then confirms that the purchase has been completed successfully.

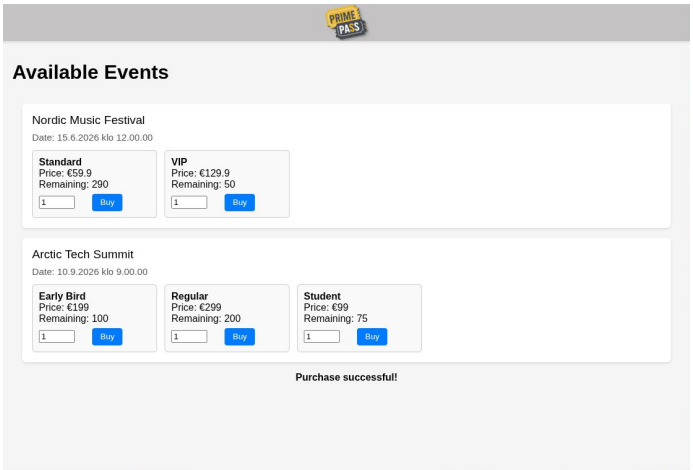

Fig. 2. Successful ticket purchase confirmation.

If a user tries to buy more tickets than are currently available, the backend rejects the request and returns an error message. The frontend displays it to the user that the requested number of tickets is more than what is in the inventory.

These results show that both successful transactions and errors are handled gracefully. The interaction between the backend ensures that a ticket is validated before a purchase is completed.

Running a concurrency test with two different payloads, one with only 20 concurrent event ticket purchases and one

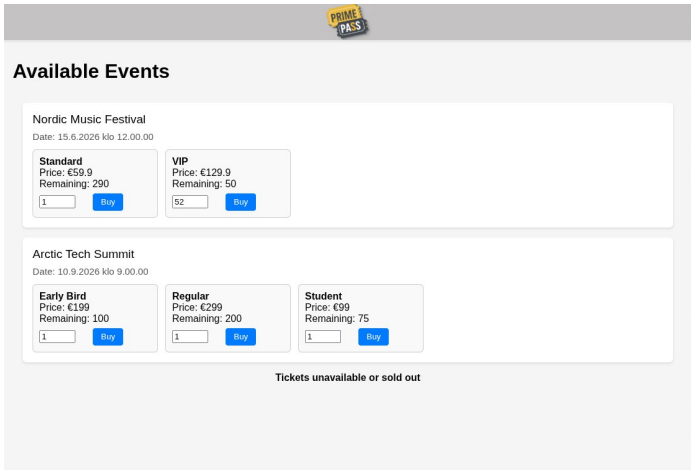

Fig. 3. Frontend message shown when insufficient tickets are available.

with 2000 concurrent purchases. The pytest results show the following:

— Concurrent Purchase Test (num_requests=20) —
Total requests: 20
Successful purchases: 10
Expected failures (sold out): 10
Unexpected failures: 0
Avg latency (success): 0.044s
Min latency: 0.029s
Max latency: 0.059s
Correct handling rate: 100.0%
Unexpected error rate: 0.0%
.
— Concurrent Purchase Test (num_requests=2000) —
Total requests: 2000
Successful purchases: 1000
Expected failures (sold out): 1000
Unexpected failures: 0
Avg latency (success): 0.049s
Min latency: 0.034s
Max latency: 0.098s
Correct handling rate: 100.0%
Unexpected error rate: 0.0%

From the results we can see that there is only slight drop-off in the average latency, while max latency shows a larger increase. As shown in Figure 4, the system latency increases as the number of concurrent requests grows. Figure 5 illustrates the distribution of successful and failed ticket purchase attempts under concurrent load.

Next we test the system on different sizes of payloads. The pytest test_payload.py measures the end-to-end response time from creating a new event to reading it results show the following:

— Event Payload Test —

Small event inserted. Tickets: 3, Response time: 0.025s, Response size: 586 bytes

Large event inserted. Tickets: 500, Response time: 0.023s, Response size: 58669 bytes

From the results we can see that there is no delay in creating an event with a small response size and only 3 ticket types

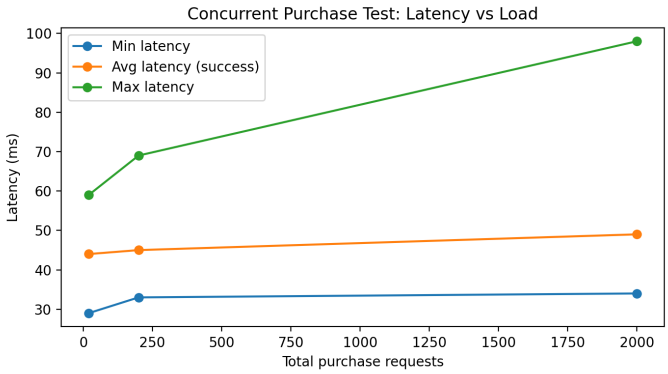

Fig. 4. Concurrency latency graph.

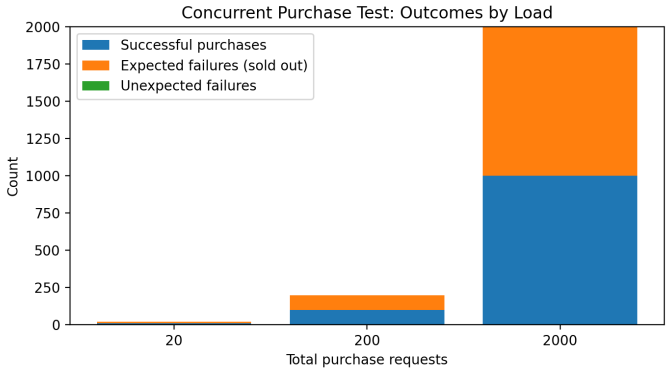

Fig. 5. Concurrency outcomes.

when compared to one with 500 ticket types and a long description.

V. Conclusion

The project has reached the minimum course project requirement as described below:

- Distributed architecture (≥3 nodes): Frontend + Backend + PostgreSQL (Primary + Replica) running as separate Kubernetes pods/services.
- Clearly defined roles: communication flowing from the browser to the frontend (Nginx), then to the backend (Flask API), and finally to the database (Postgres). All inter-service communication is managed through service discovery.
- Distributed systems functionality:
  - Consistency under concurrency: single ticket decrement prevents overselling during simultaneous purchases.
  - Replication: Postgres primary–replica setup for availability and data durability.
- Deployment + reproducibility: Docker images + Kubernetes YAML manifests for repeatable setup and scaling.
- Logging/observability (basic): Kubernetes logs available per service/pod for debugging.

Role of AI tools in the project

All modeling, algorithm implementation, and evaluation experiments were designed and executed by the author. AI-assisted tools were employed for language polishing, formatting assistance, and code debugging:

- **ChatGPT (GPT-4)**[1]**:** Utilized to refine grammar, enhance clarity of exposition, and format LaTeX tables and code snippets. The logo for the application was generated using Sora[2].
- **DeepSeek V3**[3]**:** Assisted in debugging API endpoints, Docker containerization, and Kubernetes YAML file modifications. The model was also used to partially generate and refine the pytest files.
- **Claude 3.5 Sonnet**[4]**:** Consulted for incremental troubleshooting of Python dependency issues.

No AI systems were used to generate original research hypotheses, synthesize results, or replace critical technical judgment. All substantive scientific contributions, experimental designs, analyses, and conclusions reflect the independent work of the author.

Division of Labour among group members

Learning Dairy

We learn how to use Docker and Kubernetes for a large-scale project. We also learn how to implement RESTful API to make a working database management.

The project also taught us about working with Docker and Kubernetes, as well as how replication and load balancing reduces the allocated load on an individual resource.

Time Spent

- Niklas Raesalmi, Y68722761: 57 hours
- Huan Vo, 2215079: 56 hours
- Serden-Yilmaz Kose, 2215692: 56 hours
- MD. Nur-E Ferdaus, 2511766: 56 hours
- Niranjan Sreegith, 2215077: 57 hours

References

[1] https://openai.com/chatgpt
[2] https://sora.chatgpt.com/explore/images
[3] https://www.deepseek.com/en
[4] https://www.anthropic.com/claude